PART-A

1. Write an R-Program to calculate employee salary, bonus and tax

```
Salary=30000
Bouns=Salary*0.10
Tax=Salary*0.05
net_salary=Salary+Bouns-Tax
cat("Salary is:",Salary,"\n")
cat("Bouns is:",Bouns,"\n")
cat("Tax is:",Tax,"\n")
cat("net_Salary is:",net_salary)
```

Output:

```
Salary is: 30000
Bouns is: 3000
Tax is: 1500
net_Salary is: 31500
```

2. Write an R-Program to calculate student grades using if-statement.

```
name=readline("Enter Student name:")
tmarks=as.integer(readline("Enter total marks of the student:"))
if(tmarks>=850 && tmarks<=1000)
{
  cat("grade is A")
}else if(tmarks>=650 && tmarks<850)
{
  cat("grade is B")
}else if(tmarks>=450 && tmarks<650)
{
  cat("grade  is C")
}else
{
  cat("fail")
```

```
}
```

Output:

```
enter the student name:xyz
Enter total marks of the student:670
grade is B
```

3. Write an R-Program to generate an electricity bill.

```
units<-as.numeric (readline("Enter units consumed:"))
if(units<=100)
{
  bill<-units*1.50
}else if(units<=200)
{
  bill<-100*1.50+(units-100)*2.50
}else if(units<=500)
{
  bill<-100*1.50+100*2.50+(units-200)*4.00
}else
{
  bill<-100*1.50+100*2.50+300*4.00+(units-500)*6.00
}
cat("Electricity bill is=Rs.",bill)
```

Output:

```
Enter units consumed:175
Electricity bill is=Rs. 337.5
```

4. Write a R-Program to display multiplication table by using loop

```
number<-as.numeric(readline(prompt="Enter a number:"))

for(i in 1:10)
  cat(number, "x", i, "=", number * i , "\n")
```

Output:

```
Enter a number:6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

5. Write an R-Program function to calculate simple interest and compound interest.

```
si=function(p,t,r)
{
  (p*t*r)/100
}
ci=function(p,t,r)
{
 (p*(1+r/100)^t)-p
}
cat("simple interest is:",si(1000,5,2),"\n")
cat("compound interest is:",ci(1000,5,2),"\n")
```

Output:

```
simple interest is: 100
compound interest is: 104.0808
```

6. Write an R-Program to calculate mean, median and standard deviation of student marks

```
marks<-c(95,88,98,79,78)
cat("mean of the marks is:",mean(marks),"\n")
cat("median of the marks is:",median(marks),"\n")
cat("standard deviation of the marks is:",sd(marks))
```

Output:

```
mean of the marks is: 87.6
median of the marks is: 88
standard deviation of the marks is: 9.071935
```

7. Write an R-Program to analyze stock prices using vectors.

```
#weekly stock price analysis
days<-c("Monday","Tuesday","Wednesday","Thurday","Friday")
prices<-c(250,276,289,234,320)
cat("Weekly Stock Prices: \n")
stock=data.frame(Day=days, Prices<-prices)
print(stock)

cat("\n Highest price is:",max(prices),"\n")
cat("\n lowest price is:",min(prices),"\n")
cat("\n Average price is:",mean(prices),"\n")
cat("\n Total price is:",sum(prices),"\n")
cat("\n Range of the prices is:",max(prices)-min(prices),"\n")

if(prices[5]>prices[1])
{
  cat("stock price increased in this weak \n")
}else
{
  cat("stock price decreased in this weak \n")
}
```

Output:

```
Weekly Stock Prices:
         Day Prices....prices
1     Monday                250
2    Tuesday                276
3  Wednesday                289
4   Thurday                 234
5     Friday                320

 Highest price is: 320

 lowest price is: 234

 Average price is: 273.8

 Total price is: 1369

 Range of the prices is: 86
stock price increased in this weak
```

8. Write a R-Program to store hospital patient details using list

```
patient<-list(pid=101,pname="ajith", age=50,gender="male",
              doctor="mr.Sharma", DaysAdmitted=8,
              disease="Dengue", dailycharge=3000)


patient$totalbill = patient$DaysAdmitted * patient$dailycharge
cat(".......patient detais....... \n")
cat("patient id:", patient$pid,"\n")
cat("patient Name:", patient$pname,"\n")
cat("patient age:", patient$age,"\n")
cat("gender is:", patient$gender,"\n")
cat("consulted doctor is:", patient$doctor,"\n")
cat("Patient disease is:",patient$disease,"\n")
cat("Number of days admitted is:",patient$DaysAdmitted,"\n")
cat("daily_charge:",patient$dailycharge,"\n")
cat("Total bill is:",patient$totalbill,"\n")

if(patient$age>=60)
{
  cat("senior citizen \n")
}else
{
  cat("general citizen \n")
}
```

Output:

```
.......patient detais.......
patient id: 101
patient Name: ajith
patient age: 50
gender is: male
consulted doctor is: mr.Sharma
Patient disease is: Dengue
Number of days admitted is: 8
daily_charge: 3000
Total bill is: 24000
general citizen
```

9. Write a R-Program to calculate product sales using matrix.

```
sales<-matrix(c(100,120,150,80,90,110,200,180,220),nrow=3,byrow=T)

#product names

products<-c("Product-A","Product-B","Product-C")

#month names

months<-c("January","February","March")

#assign row and column names

rownames(sales)<-products

print("sales matrix:")

print(sales)

prodtot<-rowSums(sales)

print(prodtot)

monthtot<-colSums(sales)

print(monthtot)

totalsales<-sum(sales)

print(totalsales)
```

Output:

```
[1] "sales matrix:"
           [,1] [,2] [,3]
Product-A  100  120  150
Product-B   80   90  110
Product-C  200  180  220
Product-A Product-B Product-C
      370       280       600
[1] 380 390 480
[1] 1250
```

10. Write a R-Program to store weather data using arrays.

```
Cities<-c("Tumkur", "Banglore")

Days<-c("Monday", "Tuesday", "Wednesday")
```

```
weather<-array(c(30,32,31,28,29,30),dim=c(2,3),
        dimnames=list(city=Cities,Day=Days))
print(weather)
cat("\n Highest temperature:",max(weather),"\n")
cat("\n Lowest temperature:",min(weather),"\n")
cat("\n Average temperature:",mean(weather),"\n")
cat("\n Temperature in Banglore on tuesday:",
weather["Banglore","Tuesday"],"\n")
```

Output:

```
            Day
city         Monday Tuesday Wednesday
  Tumkur         30       31         29
  Banglore       32       28         30

 Highest temperature: 32

 Lowest temperature: 28

 Average temperature: 30

 Temperature in Banglore on tuesday: 28
```

PART-B

1. Write an R-Program to categorize customer feedback using factors.

```
feedback<-c("Excellent","Poor","Good","Average","Poor",
"Excellent","Average","Average","Poor")
feedback_factor<-factor(feedback,
                levels=c("Poor","Average","Good","Excellent"),
                ordered=TRUE)
print(feedback_factor)
#frequency of level
```

```
print(table(feedback_factor))
#levels of factor
print(levels(feedback_factor))
cat("\n highest feedback category is:",max(feedback_factor),"\n")
cat("\n lowest feedback category is:",min(feedback_factor),"\n")
```

Output:

```
[1] Excellent Poor      Good      Average   Poor      Excellent Average
[8] Average   Poor
Levels: Poor < Average < Good < Excellent
feedback_factor
     Poor   Average      Good Excellent
        3         3         1         2
[1] "Poor"      "Average"   "Good"      "Excellent"

 highest feedback category is: 4

 lowest feedback category is: 1
```

2. Write an R Program to manage employee payroll using data frames.

```
employee <- data.frame(
  ID = c(101, 102, 103, 104),
  Name = c("A", "B", "C", "D"),
  Basic_Salary = c(25000, 30000, 28000, 35000)
)
# Calculate HRA (20% of Basic Salary)
employee$HRA <- employee$Basic_Salary * 0.20
# Calculate DA (10% of Basic Salary)
employee$DA <- employee$Basic_Salary * 0.10
# Calculate Gross Salary
```

```
employee$Gross_Salary <- employee$Basic_Salary + employee$HRA +
employee$DA

# Calculate PF deduction (12% of Basic Salary)

employee$PF <- employee$Basic_Salary * 0.12

# Calculate Net Salary

employee$Net_Salary <- employee$Gross_Salary - employee$PF

# Display the payroll

print(employee)
```

Output:

```
  ID Name Basic_Salary  HRA   DA Gross_Salary   PF Net_Salary
1 101    A        25000 5000 2500        32500 3000      29500
2 102    B        30000 6000 3000        39000 3600      35400
3 103    C        28000 5600 2800        36400 3360      33040
4 104    D        35000 7000 3500        45500 4200      41300
```

3. Write an R Program to import and clean CSV data.

```
# Import CSV file

data <- read.csv("Empsal.csv")

# Display the original data

print(data)

# Check for missing values

print(is.na(data))

# Count missing values in each column

print(colSums(is.na(data)))

# Remove rows containing missing values

data_clean <- na.omit(data)

# Remove duplicate rows
```

```
data_clean <- unique(data_clean)

# Display cleaned data

print(data_clean)

# Save the cleaned data into a new CSV file

write.csv(data_clean, "cleaned_empsal.csv", row.names = FALSE)
```

Output:

```
  EID Ename Department Salary
1   1   aaa         CS  12345
2   2  <NA> Mechanical  23456
3   3   ccc         IS     NA
4   4   ddd      Civil  45678
5   4   ddd      Civil  45678
       EID Ename Department Salary
[1,] FALSE FALSE      FALSE  FALSE
[2,] FALSE  TRUE      FALSE  FALSE
[3,] FALSE FALSE      FALSE   TRUE
[4,] FALSE FALSE      FALSE  FALSE
[5,] FALSE FALSE      FALSE  FALSE
       EID      Ename Department     Salary
         0          1          0          1
  EID Ename Department Salary
1   1   aaa         CS  12345
4   4   ddd      Civil  45678
```

4. Write an R Program to analyze shopping transaction data using statistical functions.

```
# Shopping transaction data

transactions <- c(250, 450, 300, 750, 500,

                  650, 400, 900, 250, 600)

# Calculate statistical values

cat("Mean =", mean(transactions),"\n")

cat("Median =", median(transactions),"\n")

cat("Minimum =", min(transactions),"\n")

cat("Maximum =", max(transactions),"\n")

cat("Variance =", var(transactions),"\n")
```

```
cat("Standard Deviation =", sd(transactions),"\n")

cat("Range =", max(transactions)-min(transactions),"\n")

tx=table(transactions)

print(tx)

mode<-which(tx==max(tx))

mode
```

Output:

```
Mean = 505
Median = 475
Minimum = 250
Maximum = 900
Variance = 48027.78
Standard Deviation = 219.1524
Range = 650
transactions
250 300 400 450 500 600 650 750 900
  2   1   1   1   1   1   1   1   1
```

5. Write an R Program to create bar chart, pie chart, histogram and line graph.

```
par(mar=c(4,4,1,1))

# Sample shopping data

months<-c("jan","feb","mar","apr","may","jun")

sales<-c(12000,15000,18000,17000,20000,22000)

# Divide the plot area into 2 rows and 2 columns

par(mfrow = c(2, 2))

# 1. Bar Chart

barplot(sales, names.arg = months, main = "Monthly Sales", xlab = "Months",

ylab = "Sales", col="lightblue")

# 2. Pie Chart
```

```
pie(sales, labels = months, main = "Sales Distribution")

 # 3. Histogram

hist(sales, main = "Sales Histogram", xlab = "Sales", ylab ="Frequency",
col="lightgreen")

# 4. Line Graph

plot(sales, type = "o", col="blue", main = "Monthly Sales", xlab = "Month",

    ylab = "Sales")
```

Output:



6. Write an R Program to identify missing values in healthcare data sets.

```
# Create a healthcare dataset

patient <- data.frame(
```

```
  Patient_ID = c(101, 102, 103, 104, 105),
  Age = c(25, 30, NA, 45, 50),
  Blood_Pressure = c(120, NA, 130, 140, NA),
  Sugar_Level = c(90, 110, NA, 125, 135),
  Cholesterol = c(180, 200, 190, NA, 220) )
# Display the dataset
print(patient)
# Check for missing values
print(is.na(patient))
# Count missing values in each column
print(colSums(is.na(patient)))
# Count total missing values
total_missing <- sum(is.na(patient))
print(total_missing)
#removing missing values
print(na.omit(patient))
```

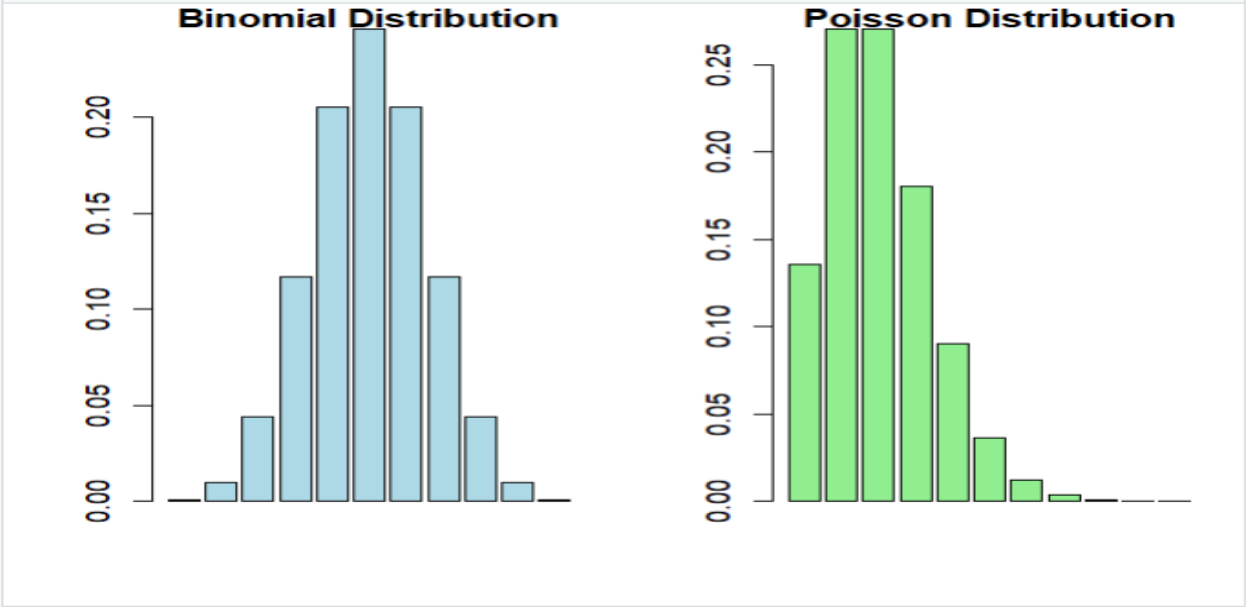
Output:

```
  Patient_ID Age Blood_Pressure Sugar_Level Cholesterol
1        101  25            120          90         180
2        102  30             NA         110         200
3        103  NA            130          NA         190
4        104  45            140         125          NA
5        105  50             NA         135         220
     Patient_ID   Age Blood_Pressure Sugar_Level Cholesterol
[1,]      FALSE FALSE          FALSE       FALSE       FALSE
[2,]      FALSE FALSE           TRUE       FALSE       FALSE
[3,]      FALSE  TRUE          FALSE        TRUE       FALSE
[4,]      FALSE FALSE          FALSE       FALSE        TRUE
[5,]      FALSE FALSE           TRUE       FALSE       FALSE
    Patient_ID            Age Blood_Pressure    Sugar_Level    Cholesterol
             0              1              2              1              1
[1] 5
  Patient_ID Age Blood_Pressure Sugar_Level Cholesterol
1        101  25            120          90         180
```

7. Write an R Program to implement Binomial and Poisson distributions.

```
#binomial distribution
cat("Binomial Distribution \n")
print(dbinom(4,size=10,prob=0.5))
#poisson distribution
cat("Poisson Distribution \n")
print(dpois(3,lambda=2))
#probability Plots
x<-0:10
par(mar=c(4,4,1,1))
par(mfrow = c(1, 2))
barplot(dbinom(x,10,0.5),main="Binomial Distribution",
        col="lightblue")
barplot(dpois(x,2),main="Poisson Distribution",
        col="lightgreen")
```

Output:



8. Write an R Program to perform t-test and Chi-square test.

```
x <- c(65, 70, 75, 80, 85)

y <- c(68, 72, 78, 79, 88)

# Perform t-test

cat("T-TEST RESULT\n")

print(t.test(x,y))

# Perform Chi-square test

data <- matrix(c(25, 15, 20, 40), nrow = 2, byrow = TRUE)

 cat("\n CHI-SQUARE TEST RESULT\n")

print(chisq.test(data))
```

Output:

```
T-TEST RESULT

        Welch Two Sample t-test

data:  x and y
t = -0.4074, df = 7.9889, p-value = 0.6944
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -13.323327   9.323327
sample estimates:
mean of x mean of y
       75        77


 CHI-SQUARE TEST RESULT

        Pearson's Chi-squared test with Yates' continuity correction

data:  data
X-squared = 7.1128, df = 1, p-value = 0.007654
```

9. Write an R Program to analyze survey data using confidence intervals.

```
# Survey data

survey_data <- c(45, 50, 48, 52, 55, 49, 51, 47, 53, 50)

# Calculate sample mean
```

```
mean_value <- mean(survey_data)

# Calculate standard deviation

sd_value <- sd(survey_data)

# Number of observations

n <- length(survey_data)

# Calculate standard error

se <- sd_value / sqrt(n)

# 95% Confidence Interval

error <- qt(0.975, df = n - 1) * se

lower_limit <- mean_value - error

upper_limit <- mean_value + error

# Display results

cat("Mean =", mean_value, "\n")

cat("Standard Deviation =", sd_value, "\n")

cat("95% Confidence Interval =", lower_limit, "to", upper_limit, "\n")
```

Output:

```
Mean = 50
Standard Deviation = 2.94392
95% Confidence Interval = 47.89405 to 52.10595
```

10. Write an R Program for house price prediction using linear regression and ggplot2.

```
library(ggplot2)

# Create sample house data
house_data <- data.frame(
  Area = c(800, 1000, 1200, 1400, 1600, 1800, 2000),
  Price = c(25, 30, 36, 42, 48, 55, 62) )
```

```
# Build Linear Regression Model
model <- lm(Price ~ Area, data = house_data)

# Display model summary
summary(model)

# Predict price for a new house
new_house <- data.frame(Area = 1500)
predicted_price <- predict(model, new_house)

cat("Predicted Price for 1500 sq.ft house =",
    predicted_price, "Lakhs\n")

# Plot using ggplot2
ggplot(house_data, aes(x = Area, y = Price)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "House Price Prediction", x = "Area (Square Feet)", y = "Price
(Lakhs)" )
```

Output:

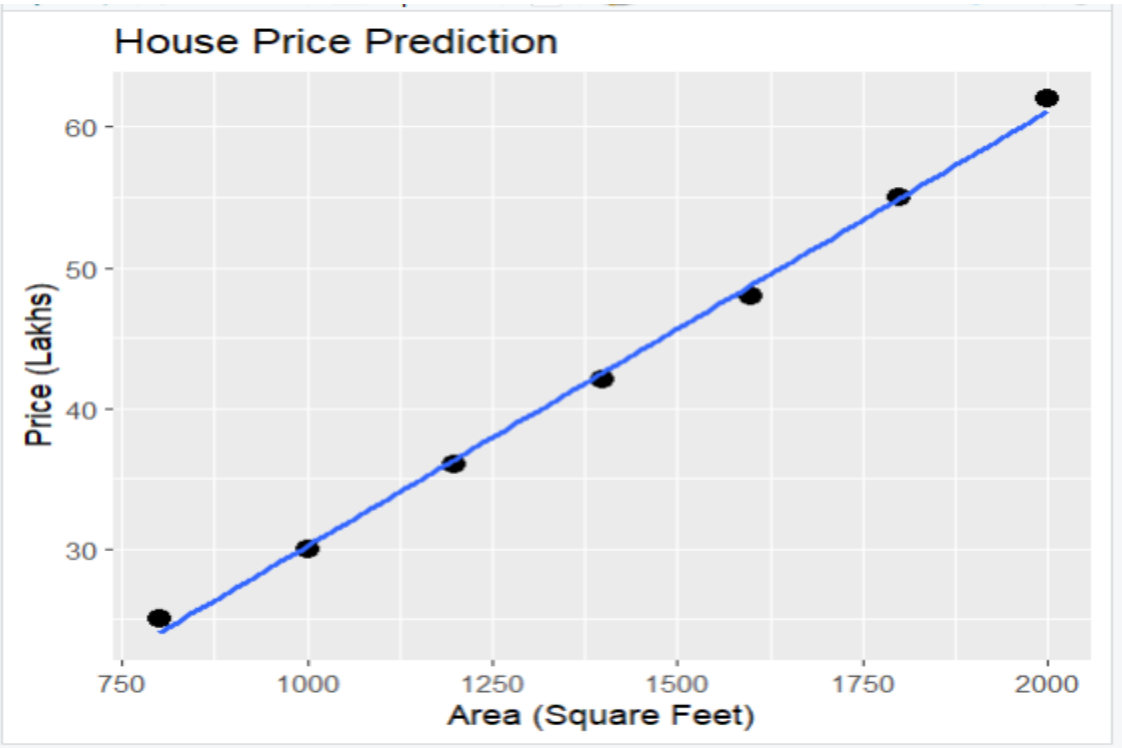
```
Call:
lm(formula = Price ~ Area, data = house_data)

Residuals:
       1        2        3        4        5        6        7
 0.96429 -0.21429 -0.39286 -0.57143 -0.75000  0.07143  0.89286

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.6785714  1.0334952  -0.657     0.54
Area         0.0308929  0.0007098  43.523 1.21e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.7512 on 5 degrees of freedom
Multiple R-squared:  0.9974,    Adjusted R-squared:  0.9968
F-statistic:  1894 on 1 and 5 DF,  p-value: 1.209e-07

Predicted Price for 1500 sq.ft house = 45.66071 Lakhs
```



- lm() — creates the linear regression model
- predict() — predicts the house price
- ggplot() — creates the graph
- geom_point() — displays the actual house-price data
- geom_smooth(method = "lm") — displays the regression line

****